```
import numpy as np
from collections import defaultdict
from time import time


def fit(self, x, y=None, tol=0.001, maxiter=20000, update_interval=140, save_dir='', cae_weights=None):
    """
    DCEC fit function modified to return cluster centroids and image directories.
    """
    # Extract image array and image directory list
    image_array, image_paths = x
    assert len(image_array) == len(image_paths), "Mismatch between image data and image paths."


    # Pretrain autoencoder (Optional, if not already pretrained)
    if cae_weights:
        self.model.load_weights(cae_weights)


    # Initialize variables
    ite = 0
    index = 0
    loss = None
    p = np.zeros((image_array.shape[0], self.n_clusters))  # Initialize cluster assignments
    y_pred_last = np.zeros(image_array.shape[0])  # Last predicted cluster


    t0 = time()
```

```
  while ite < maxiter:
    # Calculate soft cluster assignments
    p = self.model.predict(image_array, batch_size=update_interval)

    # Compute target distribution q
    q = self.target_distribution(p)

    # Training step using cluster assignments as target
    if (index + 1) * self.batch_size > image_array.shape[0]:
      loss = self.model.train_on_batch(x=image_array[index * self.batch_size::],
                       y=[p[index * self.batch_size::], image_array[index * self.batch_size::]])
      index = 0
    else:
      loss = self.model.train_on_batch(x=image_array[index * self.batch_size:(index + 1) *
self.batch_size],
                       y=[p[index * self.batch_size:(index + 1) * self.batch_size],
                        image_array[index * self.batch_size:(index + 1) * self.batch_size]])
      index += 1

    # Save intermediate models
    if ite % self.save_interval == 0:
      print(f'Saving model to: {save_dir}/dcec_model_{ite}.h5')
      self.model.save_weights(f'{save_dir}/dcec_model_{ite}.h5')

    # Check for convergence (based on predicted labels)
    delta_label = np.sum(self.y_pred != y_pred_last).astype(np.float32) /
self.y_pred.shape[0]
```

```
    y_pred_last = np.copy(self.y_pred)

    if ite > 0 and delta_label < tol:
        print(f'delta_label {delta_label} < tol {tol}')
        print('Reached tolerance threshold. Stopping training.')
        break

    ite += 1

# Save the final trained model
print(f'Saving final model to: {save_dir}/dcec_model_final.h5')
self.model.save_weights(f'{save_dir}/dcec_model_final.h5')

t3 = time()
print(f'Pretrain time: {t1 - t0}')
print(f'Clustering time: {t3 - t1}')
print(f'Total time: {t3 - t0}')

# After training, assign clusters to images
cluster_assignments = np.argmax(self.y_pred, axis=1)

# Create a dictionary to store cluster centroids and member images
clusters = defaultdict(list)
for idx, cluster_id in enumerate(cluster_assignments):
    clusters[cluster_id].append(image_paths[idx])
```

```
    # Retrieve cluster centroids
    cluster_centroids = self.model.get_layer(name='cluster').get_weights()[0]  # Assuming 'cluster' is the cluster layer name

    # Return the output in the required format: [cluster centroid, list of image dirs]
    result = []
    for i in range(cluster_centroids.shape[0]):
        centroid = cluster_centroids[i]
        members = clusters.get(i, [])
        result.append([centroid, members])

    return result
```